

THEORY AND RESEARCH

1. OWASP Top 10 Vulnerabilities

1. SQL Injection

Definition:-Exploiting a website by injecting malicious SQL statements into an input field to manipulate the database.

Example:

' OR '1'='1'; DROP TABLE users:-

Prevention Measures:

- Use parameterized queries (prepared statements).
- Validate and sanitize user inputs.
- Implement least-privileged access for the database.

2. Cross-Site Scripting (XSS)

Definition:-Injecting malicious scripts into web pages viewed by other users.

Example:

<script>alert('XSS');</script>

Prevention Measures:

- Sanitize user inputs (e.g., escaping special characters).
- Implement Content Security Policy (CSP).
- Use secure coding libraries.

3. Cross-Site Request Forgery (CSRF)

Definition:-Forcing users to execute unwanted actions on a trusted website.

Example:

A link that transfers money without the user's knowledge.

Prevention Measures:

- Use CSRF tokens.
- Implement Same Site cookies.

4. Security Misconfiguration

Definition:-Using default configurations or failing to implement security settings properly.

Example:

Running software with default credentials like admin/admin.

Regularly update software and frameworks.

Prevention Measures:

- Perform security hardening (disable unnecessary services).
- Conduct regular security audits.

5. Broken Authentication & Session Management

Definition:-Exploiting weak authentication mechanisms to hijack user sessions.

Example:

Exposed session tokens in URLs.

Prevention Measures:

- Implement MFA (Multi-Factor Authentication).
- Use secure, encrypted session management.
- Set cookies with Secure and HttpOnly flags.

6. Sensitive Data Exposure

Definition:-Exposing sensitive information like credit card numbers or personal data.

Example:

Sending passwords over HTTP instead of HTTPS.

Prevention Measures:

- Use encryption (TLS/SSL).
- Avoid storing sensitive data unnecessarily.
- Implement strong access control mechanisms.

7. Insecure Deserialization

Definition:-Exploiting vulnerabilities in deserializing objects to execute arbitrary code.

Example:

Attacker modifies serialized data to escalate privileges.

Prevention Measures:

- Avoid accepting serialized data from untrusted sources.
- Use secure deserialization libraries.

8. Using Components with Known Vulnerabilities

Definition:-Using outdated libraries or frameworks with publicly known exploits.

Example:

Running outdated versions of software like Apache or jQuery.

Prevention Measures:

- Regularly update components.
- Use vulnerability management tools like Snyk.

9. Insufficient Logging & Monitoring

Definition:-Failing to detect and log malicious activity, leading to undetected breaches.

Example:

Ignoring brute force attacks due to lack of logging.

Prevention Measures:

- Implement detailed logging mechanisms.
- Regularly review logs.
- Use security monitoring tools.

10. XML External Entity (XXE)

Definition:-Exploiting XML parsers to access sensitive data or execute harmful code.

Example:

<!DOCTYPE test [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>

Prevention Measures:

- Disable external entity processing in XML parsers.
- Use JSON instead of XML when possible

2. Explore Website Security Measures

1. CAPTCHA

Purpose:

Prevent automated bots from interacting with websites.

How It Works:

Users solve puzzles (identify objects, type characters) to prove they're human.

Example:

Google reCAPTCHA.

2. HTTPS

Purpose:

Encrypt data transmitted between the client and the server.

How It Works:

Uses TLS/SSL certificates for encryption.

Example:

URL starts with https://, ensuring secure communication.

3. Content Security Policy (CSP)

Purpose:

Prevent XSS attacks by restricting sources of content.

How It Works:

Allows only trusted sources for scripts, images, and stylesheets.

Example:

Content-Security-Policy: script-src 'self';

4. Input Validation

Purpose:

Prevent injection attacks by validating user input.

How It Works:

Restricts inputs to allowed formats and values.

5. Session Management

Purpose:

Secure user sessions and prevent hijacking.

How It Works:

Uses unique session tokens with secure attributes.

6. Web Application Firewalls (WAF)

Purpose:

Protect against attacks like SQL injection and XSS.

How It Works:

Monitors and filters incoming HTTP traffic.

7. Two-Factor Authentication (2FA)

Purpose:

Add an extra layer of security to user authentication.

How It Works:

Requires a second verification step like an OTP or biometric.

8. Security Headers

Purpose:

Protect websites from common attacks.

Examples:

- Strict-Transport-Security
- X-Frame-Options
- X-Content-Type-Options